

AA 2025-2026 - Metodi del Calcolo Scientifico - Progetto 2

Benbouzid Hicham 894680, Shi Le Yang 894536, Zen Elena 895355

Contents

1	Introduzione	2
2	Parte I	3
2.1	La Libreria JTransforms	3
2.1.1	Architettura e Algoritmo Base	3
2.1.2	Limitazioni e Vincoli Operativi	4
2.1.3	Il supporto alla Trasformata Discreta di Fourier (DFT) e stato di manutenzione	4
2.2	Analisi Prestazionale e Confronto dei Tempi di Esecuzione	4
2.2.1	Analisi e Commento dei Grafici	5
2.2.2	Listato del codice	5
3	Parte II	7
3.1	Struttura	7
3.2	Sperimentazioni	8
3.2.1	bridge.bmp	8
3.2.2	gradient.bmp	13
3.2.3	tonypitony.bmp	14
3.3	Conclusioni	16
3.4	Listato del codice	16

Chapter 1

Introduzione

Il presente documento costituisce la relazione del progetto riguardante la compressione di immagini tramite la DCT. L'obiettivo principale di questo lavoro è utilizzare un'implementazione della Trasformata Discreta del Coseno bidimensionale (**DCT2**) per studiare gli effetti di un algoritmo di compressione *simil-JPEG* su immagini in toni di grigio, **omettendo** l'utilizzo di una matrice di quantizzazione. Il progetto si articola in **due** fasi operative principali, che uniscono l'analisi prestazionale degli algoritmi allo sviluppo di un applicativo per l'elaborazione delle immagini.

La prima parte si concentra sull'implementazione della **DCT2** e sul confronto dei suoi tempi di esecuzione con quelli di una versione ottimizzata ricavata da una libreria di calcolo fast (**FFT**). Lo studio prestazionale mira a valutare i tempi di elaborazione al variare della dimensione di array quadrati $N \times N$.

La seconda parte è dedicata all'implementazione di un software dotato di una semplice interfaccia per manipolare visivamente i dati. L'applicativo consente di caricare un'immagine in formato **BMP** dal filesystem e di configurare due parametri chiave: l'ampiezza F dei macro-blocchi quadrati in cui suddividere l'immagine e la soglia d per il taglio delle frequenze. L'immagine compressa viene poi visualizzata a schermo affiancata all'originale per permettere un riscontro immediato della compressione. Nelle sezioni successive verranno descritti in dettaglio la struttura del codice open-source sviluppato e le caratteristiche della libreria utilizzata, aggiungendo anche notizie per le trasformate di Fourier.

Nota sulla validazione e correttezza dello scaling: Al fine di garantire l'affidabilità del software sviluppato e verificare la corretta calibrazione dello scaling della trasformata rispetto alle specifiche d'esame, è stato eseguito un test preliminare di validazione. Il test ha preso in input il blocco predefinito 8×8 composto dai seguenti valori interi:

$$A = \begin{bmatrix} 231 & 32 & 233 & 161 & 24 & 71 & 140 & 245 \\ 247 & 40 & 248 & 245 & 124 & 204 & 36 & 107 \\ 234 & 202 & 245 & 167 & 9 & 217 & 239 & 173 \\ 193 & 190 & 100 & 167 & 43 & 180 & 8 & 70 \\ 11 & 24 & 210 & 177 & 81 & 243 & 8 & 112 \\ 97 & 195 & 203 & 47 & 125 & 114 & 165 & 181 \\ 193 & 70 & 174 & 167 & 41 & 30 & 127 & 245 \\ 87 & 149 & 57 & 192 & 65 & 129 & 178 & 228 \end{bmatrix}$$

Applicando la trasformata DCT2 della libreria scelta, si è ottenuto il seguente risultato atteso in notazione scientifica:

$$\begin{bmatrix} 1.12e+03 & 4.40e+01 & 7.59e+01 & -1.39e+02 & 3.50e+00 & 1.22e+02 & 1.95e+02 & -1.02e+02 \\ 7.72e+01 & 1.15e+02 & -2.18e+01 & 4.14e+01 & 8.78e+00 & 9.91e+01 & 1.38e+02 & 1.09e+01 \\ 4.48e+01 & -6.28e+01 & 1.12e+02 & -7.64e+01 & 1.24e+02 & 9.56e+01 & -3.98e+01 & 5.85e+01 \\ -7.00e+01 & -4.02e+01 & -2.35e+01 & -7.67e+01 & 2.66e+01 & -3.68e+01 & 6.62e+01 & 1.25e+02 \\ -1.09e+02 & -4.33e+01 & -5.55e+01 & 8.17e+00 & 3.03e+01 & -2.87e+01 & 2.44e+00 & -9.41e+01 \\ -5.39e+00 & 5.66e+01 & 1.73e+02 & -3.54e+01 & 3.24e+01 & 3.35e+01 & -5.81e+01 & 1.90e+01 \\ 7.88e+01 & -6.46e+01 & 1.19e+02 & -1.51e+01 & -1.37e+02 & -3.06e+01 & -1.05e+02 & 3.98e+01 \\ 1.98e+01 & -7.82e+01 & 9.72e-01 & -7.23e+01 & -2.16e+01 & 8.13e+01 & 6.37e+01 & 5.91e+00 \end{bmatrix}$$

L'esito positivo di questo test preliminare e la coerenza dei risultati ottenuti con i dati di riferimento confermano il corretto funzionamento della libreria di calcolo integrata e l'adequatezza del fattore di normalizzazione applicato.

Chapter 2

Parte I

La prima parte del progetto è incentrata sullo studio prestazionale della Trasformata Discreta del Coseno bidimensionale, ponendo a confronto un'implementazione sviluppata direttamente da **zero** con la versione **ottimizzata** di riferimento. L'obiettivo principale consiste nell'analizzare empiricamente l'andamento dei tempi di calcolo al variare della dimensione N di array quadrati $N \times N$, verificandone la coerenza con le complessità asintotiche teoriche.

Per soddisfare il requisito di utilizzo di un ambiente open source, lo sviluppo del codice è stato interamente realizzato in **Java**. La versione "fast" della trasformata, configurata come termine di paragone prestazionale, è stata scelta di utilizzare la libreria open source **JTransforms**, un'estensione in Java dedicata al calcolo efficiente delle trasformate di *Fourier* e del *cosenoidi*.

L'analisi sperimentale prevede la misurazione dei tempi di esecuzione per valori crescenti di N . I dati raccolti verranno proiettati su un grafico in scala semilogaritmica (esclusivamente lungo l'asse delle ordinate) per valutare la corrispondenza con i modelli teorici, che prevedono tempi proporzionali a N^3 per l'algoritmo sviluppato manualmente e a $N^2 \log(N)$ per l'algoritmo ottimizzato della libreria, isolando e commentando eventuali fluttuazioni o andamenti irregolari propri delle librerie fast.

2.1 La Libreria JTransforms

Per l'implementazione della versione ottimizzata ("fast") della DCT2 si è scelto di utilizzare **JTransforms**[1]. Si tratta della **prima** libreria open-source per il calcolo della Trasformata Rapida di Fourier (**FFT**) scritta interamente in Java puro e ingegnerizzata per supportare nativamente il **multithreading**. Sviluppata originariamente da Piotr Wendykier e derivata dal pacchetto **General Purpose FFT Package** (OouraFFT) di Takuya Ooura, JTransforms garantisce prestazioni computazionali elevate e del tutto competitive rispetto alle classiche implementazioni ottimizzate in linguaggio C, come la nota libreria di riferimento FFTW.

2.1.1 Architettura e Algoritmo Base

Il nucleo computazionale di JTransforms si basa sul modello matematico dell'algoritmo **split-radix**. Dal punto di vista algebrico, questo approccio divide in modo **ricorsivo** una trasformata discreta di dimensione N in una sotto-trasformata di dimensione $N/2$ e due sotto-trasformate di dimensione $N/4$. Questa scomposizione strategica consente di **ridurre** al minimo il conteggio complessivo delle operazioni aritmetiche necessarie al calcolo quando la dimensione dell'array è una potenza di due. Rispetto alla libreria nativa in C da cui trae origine, JTransforms introduce significative ottimizzazioni architetturali strettamente legate alla piattaforma Java:

- JTransforms integra l'infrastruttura dei *cached thread pool* introdotta a partire da Java 5. Questo meccanismo permette il riutilizzo **dinamico** di thread precedentemente allocati non appena ritornano disponibili, abbattendo l'overhead computazionale associato alla continua creazione e distruzione dei thread tipica di molte librerie concorrenti in ambiente C.
- La libreria implementa un sistema di parallelismo trasparente, a differenza di pacchetti come *FFTW* o *OouraFFT*, che richiedono la configurazione manuale e statica del numero **massimo** di thread computazionali, JTransforms rileva autonomamente le risorse hardware (CPU e core fisici) disponibili sulla macchina ospite all'atto dell'esecuzione, adattando di conseguenza il carico **parallelo**.
- L'interfaccia di programmazione della libreria si presenta estremamente *snella*. Non prevede alcuna complessa fase di pianificazione preliminare; allo sviluppatore è richiesto **unicamente** di specificare la dimensione del dato in input, mentre gli array ausiliari di lavoro vengono allocati e gestiti internamente in modo del tutto trasparente.

2.1.2 Limitazioni e Vincoli Operativi

Nonostante l'elevata efficienza prestazionale su architetture multicore, la stretta **dipendenza** dall'algoritmo split-radix impone alcuni vincoli strutturali rigidi che devono essere opportunamente considerati in fase di progettazione del software:

1. **Vincolo geometrico sulle dimensioni** (Potenze di due): La lunghezza e le dimensioni dei dati di input devono essere **tassativamente** una potenza di due. Qualora l'applicazione si trovi a elaborare matrici o immagini con dimensioni arbitrarie (come numeri primi o composti da fattori primi grandi), la libreria non può operare direttamente. In tali contesti si rende necessario implementare una fase di pre-elaborazione dei dati tramite tecniche di **padding** (es. reflexive padding o zero padding) per estendere artificialmente l'array alla potenza di due immediatamente successiva.
2. **Vincoli sul parallelismo**: Il numero di thread computazionali allocati per l'esecuzione deve essere anch'esso una potenza di **due**. Inoltre, le trasformate **monodimensionali** (1D) presentano una limitazione interna che restringe l'uso del multithreading a un massimo di due o quattro thread concorrenti.

2.1.3 Il supporto alla Trasformata Discreta di Fourier (DFT) e stato di manutenzione

Oltre alle funzionalità specifiche per il calcolo della trasformata del coseno, la libreria JTransforms offre un supporto nativo e ottimizzato per la Trasformata Discreta di Fourier (**DFT**, *Discrete Fourier Transform*). La DFT rappresenta uno strumento fondamentale nell'elaborazione digitale dei segnali e delle immagini: il suo scopo è convertire un insieme di dati discreti dal dominio cinematico o spaziale (come i pixel di un'immagine) al dominio delle **frequenze**. Questo passaggio permette di scomporre il segnale originale nelle sue componenti armoniche (onde sinusoidali complesse), rendendo esplicito il contenuto in frequenza del dato.

Un aspetto cruciale nella scelta di questo strumento per lo sviluppo del progetto riguarda l'**affidabilità** e la **longevità** del software. JTransforms si distingue per essere una libreria ampiamente collaudata e adeguatamente mantenuta nel tempo. Sebbene il nucleo originale risalga ai primi studi sulle trasformate parallele in ambiente Java, il progetto ha continuato a ricevere un supporto costante da parte della comunità open-source. In particolare, a partire dal 2014, la libreria viene regolarmente aggiornata con una cadenza di circa 1 o 2 anni. Questo ciclo di manutenzione continua garantisce la stabilità del codice, la risoluzione di eventuali bug e la perfetta **compatibilità** con le versioni più recenti del Java Development Kit (JDK), rendendola una soluzione solida per applicazioni di calcolo scientifico.

2.2 Analisi Prestazionale e Confronto dei Tempi di Esecuzione

In questa sezione viene analizzato il comportamento prestazionale della DCT2 sviluppata manualmente, ponendola a diretto confronto con la versione ottimizzata fornita dalla libreria **JTransforms**. L'obiettivo dell'esperimento consiste nel verificare empiricamente la rispondenza dei due approcci alle rispettive complessità asintotiche teoriche al variare della dimensione N .

Le sessioni di benchmarking sono state condotte interamente all'interno dell'ambiente operativo **Windows**, misurando accuratamente il tempo di calcolo in **millisecondi** richiesto per completare la trasformata. Per evidenziare la divergenza tra i due algoritmi, i dati raccolti sono stati proiettati su un grafico in scala semilogaritmica applicata esclusivamente all'asse delle ordinate.

I valori numerici specifici durante le prove di carico per i diversi valori di N campionati sono riportati nel file **.csv** sulla repository GitHub.

2.2.1 Analisi e Commento dei Grafici

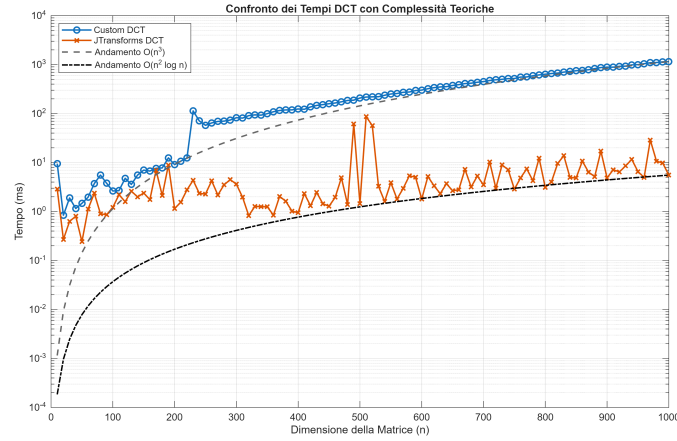


Figure 2.1: Confronto dei tempi

L'andamento delle curve rappresentate nel grafico semilogaritmico permette di valutare con chiarezza la separazione netta tra le **efficienze** dei due algoritmi all'espandersi del dominio di calcolo:

- **Algoritmo DCT2 Custom:** Mostra una crescita dei tempi estremamente rapida. All'interno della rappresentazione semilogaritmica, la curva tende a flettere verso l'alto con un'inclinazione pronunciata, rispecchiando l'andamento cubico teorico pari a $\mathcal{O}(N^3)$. Tale comportamento è determinato dai cicli iterativi annidati necessari per mappare la trasformazione diretta senza scomposizioni ottimizzate del segnale.
- **Libreria JTransforms:** Mantiene i tempi di calcolo confinati su ordini di grandezza sensibilmente inferiori, evidenziando l'efficacia della variante fast basata sull'algoritmo split-radix. La progressione asintotica si dimostra coerente con il modello teorico stimato in $\mathcal{O}(N^2 \log N)$.

Come preventivato dalle note metodologiche del progetto, la curva prestazionale associata a JTransforms non presenta una crescita perfettamente uniforme o rigidamente lineare sulla scala logaritmica, ma manifesta un andamento localmente irregolare. Questa variabilità è una caratteristica intrinseca degli algoritmi di tipo FFT a causa di molteplici fattori architetturali e sistemistici:

1. **Overhead del Thread Pool:** Per valori di N ridotti, i tempi associati al risveglio e alla sincronizzazione dei thread concorrenti all'interno del pool di Java possono incidere proporzionalmente più del calcolo aritmetico vero e proprio, stabilizzando i tempi verso l'alto.
2. **Fattorizzazione della dimensione:** L'algoritmo ottimizza in modo estremo le operazioni quando le dimensioni corrispondono a potenze perfette di due, introducendo fluttuazioni minori qualora i passaggi interni della scomposizione ricorsiva richiedano passaggi di allineamento o padding dei dati.

In conclusione, i test eseguiti confermano l'adeguatezza dell'algoritmo sviluppato da zero nel rispettare i vincoli della definizione matematica fondamentale, evidenziando al contempo come l'adozione di un motore di calcolo strutturato come JTransforms risulti imprescindibile per l'elaborazione efficiente di domini bidimensionali complessi o immagini ad alta risoluzione.

2.2.2 Listato del codice

```
1 package org.example.custom_dct;
2
3 import org.ejml.simple.SimpleMatrix;
4
5 public class DCT {
6
7     /**
8      * Calcola la matrice D utilizzata per la DCT. La matrice D e' una matrice di dimensione n x n
9      * che contiene i coefficienti di coseno necessari per calcolare la DCT.
10     * La matrice D viene calcolata utilizzando la formula:
11     * D[k][j] = cos(pi * k * (2 * j + 1) / (2 * n)) * alpha
12     * dove alpha e' una costante che dipende da k e n. Se k e' 0, alpha e' 1 / sqrt(n),
13     * altrimenti alpha e' sqrt(2 / n).
14     *
15     * @param n Dimensione della matrice D da calcolare
```

```

16     * @return Matrice D calcolata
17     */
18     public SimpleMatrix computeD(int n){
19         SimpleMatrix d = new SimpleMatrix(n,n);
20         double alpha;
21         for(int k = 0; k < n; k++){
22             if(k == 0){
23                 alpha = 1 / Math.sqrt(n);
24             } else {
25                 alpha = Math.sqrt((double) 2 / n);
26             }
27             for(int j = 0; j < n; j++){
28                 d.set(k, j, Math.cos(Math.PI * k * (2 * j + 1) / (2 * n)) * alpha);
29             }
30         }
31         return d;
32     }
33
34     /**
35     * Calcola la DCT di una matrice utilizzando la matrice D calcolata con il metodo computeD.
36     *
37     * @param matrix Matrice di input per la quale calcolare la DCT
38     * @param n Numero di righe della matrice di input
39     * @param m Numero di colonne della matrice di input
40     * @param dn Matrice D calcolata per le righe della matrice di input
41     * @param dm Matrice D calcolata per le colonne della matrice di input
42     * @return Matrice DCT calcolata
43     */
44     private SimpleMatrix computeDct(SimpleMatrix matrix, int n, int m, SimpleMatrix dn,
SimpleMatrix dm) {
45         for(int j = 0; j < n; j++){
46             // moltiplicate dm to the row j of c and store the result in the row j of c
47             matrix.setRow(j, 0, dm.mult(matrix.extractVector(true, j).transpose()).getDDRM().
getData());
48         }
49         for(int l = 0; l < m; l++){
50             // moltiplicate dn to the column l of c and store the result in the column l of c
51             matrix.setColumn(l, 0, dn.mult(matrix.extractVector(false, l)).getDDRM().getData());
52         }
53         return matrix;
54     }
55
56     /**
57     * Calcola la DCT bidimensionale di una matrice.
58     *
59     * @param f Matrice di input per la quale calcolare la DCT
60     * @return Matrice DCT calcolata
61     */
62     public SimpleMatrix dct2(SimpleMatrix f){
63         SimpleMatrix c = f.copy();
64         int n = f.getNumRows();
65         int m = f.getNumCols();
66         SimpleMatrix dn = computeD(n);
67         SimpleMatrix dm = computeD(m);
68
69         return computeDct(c, n, m, dn, dm);
70     }
71
72     /**
73     * Calcola la DCT inversa di una matrice. La DCT inversa viene calcolata utilizzando la
74     * matrice D calcolata con il metodo computeD, ma trasposta.
75     *
76     * @param c Matrice di input per la quale calcolare la DCT inversa
77     * @return Matrice DCT inversa calcolata
78     */
79     public SimpleMatrix idct2(SimpleMatrix c){
80         SimpleMatrix f = c.copy();
81         int n = c.getNumRows();
82         int m = c.getNumCols();
83         SimpleMatrix dn = computeD(n).transpose();
84         SimpleMatrix dm = computeD(m).transpose();
85
86         return computeDct(f, n, m, dn, dm);
87     }
88 }

```

Listing 2.1: DCT2 Custom

Chapter 3

Parte II

La seconda parte del progetto è interamente dedicata alla progettazione e allo sviluppo di un applicativo software finalizzato alla compressione di immagini digitali in toni di **grigio**. L'obiettivo principale consiste nello studiare gli effetti visivi e quantitativi di un algoritmo di compressione simil-JPEG, basato sulla rimozione **selettiva** delle alte frequenze nel dominio trasformato e omettendo l'impiego di una matrice di quantizzazione standard.

3.1 Struttura

L'interfaccia è realizzata in Java usando Swing/AWT e impacchettata con Maven (pom.xml); oltre alle librerie standard UI, il progetto utilizza dipendenze per I/O e manipolazione immagini. Per la DCT è stata utilizzata la libreria *JTransforms*, già impiegata per la prima parte del progetto.

- **Build:** pom.xml: file Maven per build e dipendenze.
- **Risorse immagini:** images: directory contenente immagini di input e risorse usate dall'app.
- **Codice core:** core: contiene DctService, ImageService, DctTest: logica DCT, manipolazione immagini e classi di test/demo.
- **Interfaccia utente:** ui: contiene DctApp, DctController: UI Swing e controller che gestisce eventi e interazioni.
- **Output compilato:** target: classi compilate e artefatti di build.

Flusso principale

- **Avvio:** la classe *DctApp* è l'entry point (contiene *main*) e costruisce l'interfaccia.
- **Controllo:** *DctController* riceve eventi UI (carica, applica, salva) e coordina le operazioni.
- **Elaborazione:** le operazioni di trasformato e manipolazione immagine sono delegate a *DctService* e *ImageService*.
- **Persistenza/risorse:** le immagini vengono lette e salvate dalla UI tramite i servizi e le risorse nella cartella *images*.

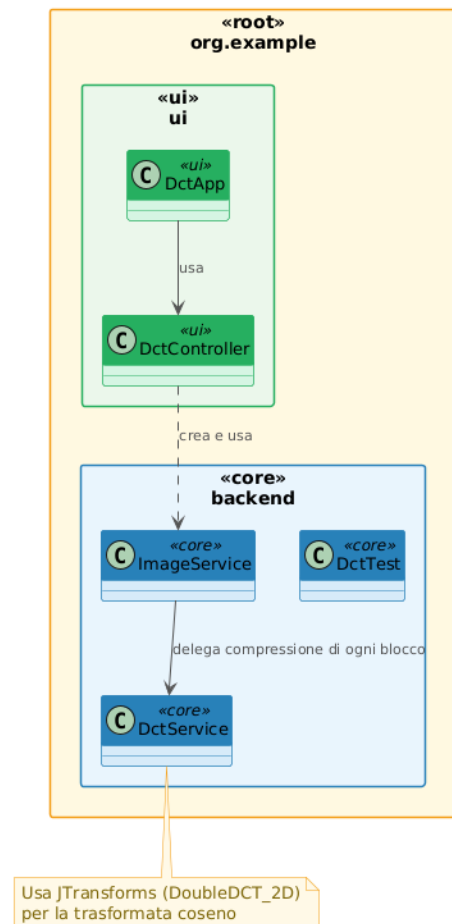


Figure 3.1: Struttura del progetto

3.2 Sperimentazioni

Sono stati condotti 20 test (variando i valori di F e d) su tre immagini in formato BMP: 7 per bridge.bmp, 5 per gradient.bmp e 8 per tonypitony.bmp. Nelle sezioni sottostanti sono stati collocati i risultati.

3.2.1 bridge.bmp



Figure 3.2: Immagine originale di bridge.bmp

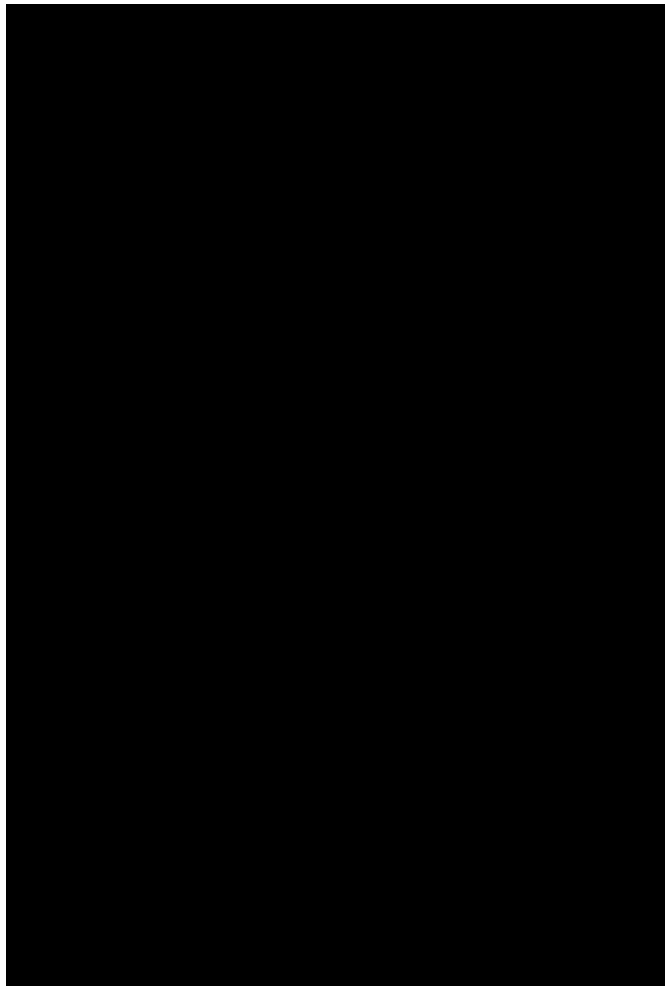


Figure 3.3: Compressione di bridge.bmp con $F = 8$ e $d = 0$

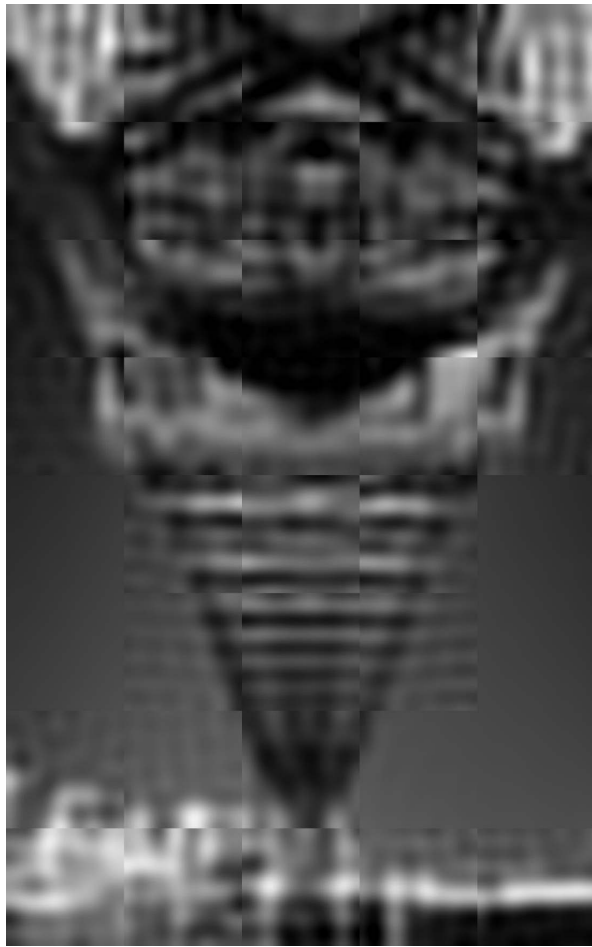


Figure 3.4: Compressione di bridge.bmp con $F = 500$ e $d = 10$



Figure 3.5: Compressione di bridge.bmp con $F = 500$ e $d = 50$



Figure 3.6: Compressione di bridge.bmp con $F = 500$ e $d = 500$

Nel primo test, ponendo la soglia di taglio d a 0, le frequenze sono state tutte azzerate e di conseguenza, l'immagine compressa è tutta nera ($k + l \geq 0$ vale per tutte le celle della matrice).

Nel secondo test F è stato posto a 500 e quindi i possibili valori della d vanno da 0 a 998:

- $d = 10$: dato che la dimensione dell'immagine è 2749×4049 e ogni blocco ha dimensione 500, l'immagine viene divisa in 40 blocchi. Essendo che il valore della d è basso, il risultato viene sgranato con i blocchi di dimensione 500 visibili.
- $d = 50$: aumentando il valore di d , aumenta anche la qualità dell'immagine risultante, rimanendo però più sgranata dell'originale.
- $d = 500$: con questo valore che si colloca a metà tra i valori validi della d , l'immagine compressa si avvicina alla qualità dell'originale e ad occhio nudo sembra la stessa.

3.2.2 gradient.bmp

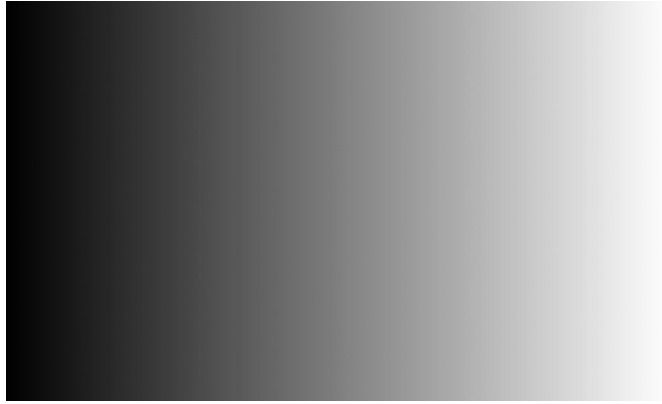


Figure 3.7: Immagine originale di gradient.bmp

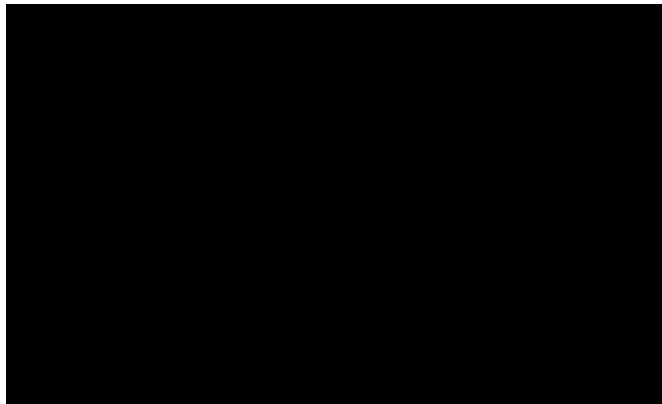


Figure 3.8: Compressione di gradient.bmp con $F = 8$ e $d = 0$

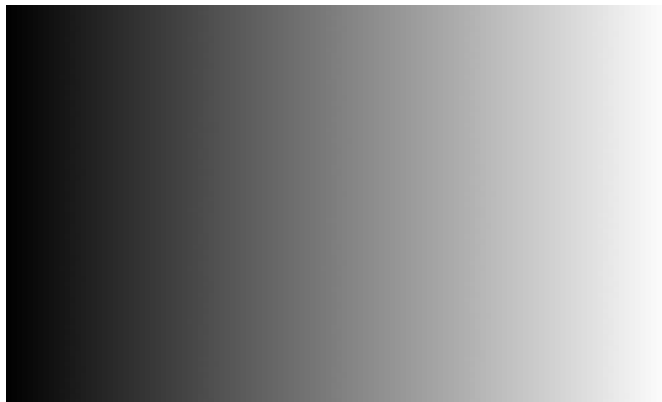


Figure 3.9: Compressione di gradient.bmp con $F = 8$ e $d = 1$

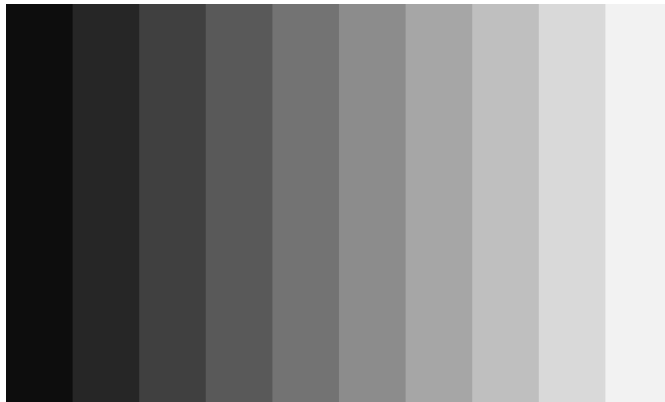


Figure 3.10: Compressione di gradient.bmp con $F = 100$ e $d = 1$



Figure 3.11: Compressione di gradient.bmp con $F = 100$ e $d = 2$

Anche in questo caso ponendo d a 0, si è ottenuto il risultato atteso, ovvero l'immagine compressa tutta nera. Nel primo test la dimensione di ogni blocco F è stata posta a 8 e dato che l'immagine ha dimensione 1000x600 e $d = 1$ (valore piccolo rispetto al massimo di 14), l'immagine risultante è leggermente sgranata e osservando attentamente si possono notare tanti quadratini piccoli di dimensione 8. Con $F = 100$, invece, l'immagine viene divisa in 10 verticalmente, questo è dovuto al fatto che spostandosi orizzontalmente si nota un cambiamento della tonalità, cosa che non succede spostandosi verticalmente. Infatti, si notano solo le divisioni verticali e non quelle orizzontali:

- $d = 1$: la soglia di taglio è molto bassa, quindi l'immagine compressa è molto sgranata e si notano bene le 10 divisioni dell'immagine.
- $d = 2$: la qualità dell'immagine compressa è aumentata ma si notano ancora le divisioni.

3.2.3 tonypitony.bmp



Figure 3.12: Immagine originale di tonypitony.bmp



Figure 3.13: Compressione di tonypitony.bmp con $F = 8$ e $d = 0$



Figure 3.14: Compressione di tonypitony.bmp con $F = 8$ e $d = 14$



Figure 3.15: Compressione di tonypitony.bmp con $F = 58$ e $d = 10$



Figure 3.16: Compressione di tonypitony.bmp con $F = 58$ e $d = 97$



Figure 3.17: Compressione di tonypitony.bmp con $F = 400$ e $d = 100$



Figure 3.18: Compressione di tonypitony.bmp con $F = 400$ e $d = 700$

Oltre ad aver ripetuto il solito test con $d = 0$, con $F = 8$ è stato posto d a 14, ovvero il suo valore massimo possibile e infatti, l'immagine risultante è molto chiara.

Con $F = 58$, la d può valere da 0 a 114:

- $d = 10$: con una soglia di taglio bassa, l'immagine compressa risulta giustamente sfocata.
- $d = 97$: con una soglia che si avvicina al valore massimo viene molto chiara e quasi uguale all'originale ad occhio nudo.

Con $F = 400$, la d può valere da 0 a 798:

- $d = 100$: dato che l'immagine è 808x450 e la dimensione di ogni blocco è 400, si nota un taglio al centro dato che l'immagine è stata divisa in due.
- $d = 700$: anche in questo caso, con un valore che si avvicina al massimo, l'immagine compressa ha una qualità molto alta.

3.3 Conclusioni

In sintesi, gli esperimenti condotti hanno restituito i risultati attesi. Le sessioni di test hanno permesso di coprire una buona parte degli scenari operativi previsti, confermando empiricamente che il codice implementato risulta pienamente funzionale, stabile e coerente con gli obiettivi prefissati dal progetto.

3.4 Listato del codice

```

1 package org.example.core;
2
3 import java.awt.Color;
4 import java.awt.Graphics2D;
5 import java.awt.image.BufferedImage;
6 import java.io.File;
7 import java.io.IOException;
8
9 import javax.imageio.ImageIO;
10
11 public class ImageService {

```

```

12
13 private final DctService dctService;
14
15 public ImageService() {
16     this.dctService = new DctService();
17 }
18
19 /**
20  * Elabora l'immagine originale e restituisce quella compressa.
21  *
22  * @param imageFile Il file originale
23  * @param F          La dimensione dei macro-blocchi
24  * @param d          La soglia di taglio
25  * @return L'immagine BufferedImage processata
26  * @throws IOException Se ci sono problemi nella lettura del file
27  */
28 public BufferedImage processImage(File imageFile, int F, int d) throws IOException {
29     // Leggi l'immagine
30     BufferedImage rawImg = ImageIO.read(imageFile);
31
32     // Disegniamo l'immagine su un nuovo canvas TYPE_INT_RGB per forzare
33     // una corretta interpretazione del colore ed evitare il bug di getRGB()
34     BufferedImage originalImg = new BufferedImage(rawImg.getWidth(), rawImg.getHeight(),
35     BufferedImage.TYPE_INT_RGB);
36     Graphics2D g2 = originalImg.createGraphics();
37     g2.drawImage(rawImg, 0, 0, null);
38     g2.dispose();
39
40     int origWidth = originalImg.getWidth();
41     int origHeight = originalImg.getHeight();
42
43     // Calcola le nuove dimensioni scartando gli avanzzi (multipli esatti di F)
44     int newWidth = (origWidth / F) * F;
45     int newHeight = (origHeight / F) * F;
46
47     // Crea una nuova immagine vuota per i risultati in toni di grigio
48     BufferedImage resultImg = new BufferedImage(newWidth, newHeight, BufferedImage.
49     TYPE_INT_RGB);
50
51     // Scorri l'immagine a blocchi di F x F
52     for (int y = 0; y < newHeight; y += F) {
53         for (int x = 0; x < newWidth; x += F) {
54
55             // 1. Estrai il blocco
56             double[][] block = new double[F][F];
57             for (int i = 0; i < F; i++) {
58                 for (int j = 0; j < F; j++) {
59                     // Estrai il valore del pixel.
60                     // Usiamo Color per prendere il canale Red (essendo scala di grigi R=G=B)
61                     Color color = new Color(originalImg.getRGB(x + j, y + i));
62                     block[i][j] = color.getRed();
63                 }
64             }
65
66             // 2. Comprimi il blocco usando il DctService
67             int[][] processedBlock = dctService.compressBlock(block, F, d);
68
69             // 3. Ricomponi il blocco nell'immagine finale
70             for (int i = 0; i < F; i++) {
71                 for (int j = 0; j < F; j++) {
72                     int grayValue = processedBlock[i][j];
73
74                     // Ricrea il colore RGB a partire dal singolo valore di grigio
75                     // Il formato e' Alpha-Red-Green-Blue
76                     int rgb = new Color(grayValue, grayValue, grayValue).getRGB();
77                     resultImg.setRGB(x + j, y + i, rgb);
78                 }
79             }
80
81         }
82     }
83
84     return resultImg;
85 }

```

Listing 3.1: Classe principale ImageService

```

1  package org.example.core;
2
3  import org.jtransforms.dct.DoubleDCT_2D;
4
5  public class DctService {
6
7      /**
8       * Comprime un singolo blocco F x F.
9       *
10      * @param f La matrice dei pixel del blocco originale
11      * @param F La dimensione del blocco
12      * @param d La soglia di taglio delle frequenze
13      * @return La matrice dei pixel del blocco compresso e normalizzato
14      */
15      public int[][] compressBlock(double[][] f, int F, int d) {
16          // 1. Inizializza la DCT2 per una matrice F x F
17          DoubleDCT_2D dct2 = new DoubleDCT_2D(F, F);
18
19          // 2. Calcola c = DCT2(f).
20          // Il parametro 'true' applica lo scaling ortonormale.
21          dct2.forward(f, true);
22
23          // 3. Taglio delle frequenze: azzeri i coefficienti con k + l >= d
24          for (int k = 0; k < F; k++) {
25              for (int l = 0; l < F; l++) {
26                  if (k + l >= d) {
27                      f[k][l] = 0.0;
28                  }
29              }
30          }
31
32          // 4. Applica la DCT2 inversa: ff = IDCT2(c)
33          dct2.inverse(f, true);
34
35          // 5. Arrotondamento e saturazione per byte (0-255)
36          int[][] ff = new int[F][F];
37          for (int k = 0; k < F; k++) {
38              for (int l = 0; l < F; l++) {
39                  // Arrotonda all'intero piu' vicino
40                  long rounded = Math.round(f[k][l]);
41
42                  // Metti a zero i valori negativi e a 255 quelli maggiori di 255
43                  if (rounded < 0) {
44                      ff[k][l] = 0;
45                  } else if (rounded > 255) {
46                      ff[k][l] = 255;
47                  } else {
48                      ff[k][l] = (int) rounded;
49                  }
50              }
51          }
52
53          return ff;
54      }
55  }

```

Listing 3.2: Classe DctService che viene usata per ogni blocco di dimensione F

Bibliography

- [1] Piotr Wendykier and James G. Nagy. Large-scale image deblurring in java. In Marian Bubak, Geert Dick van Albada, Jack Dongarra, and Peter M. A. Sloot, editors, Computational Science – ICCS 2008, pages 721–730, Berlin, Heidelberg, 2008. Springer Berlin Heidelberg. ISBN 978-3-540-69384-0.